

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:100

Annexure A

Experiments

Code of Conduct:

I C. Sai Harshitha (192424346) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

C. Sai Harshitha

Annexure A

1. **Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.**

Aim:

To implement a Naïve Bayes classifier on a real-world dataset using different feature sets and compare its performance with and without Laplace smoothing using accuracy, precision, and recall.

Dataset:

Breast Cancer Wisconsin Dataset

- 569 samples
- 30 features
- 2 classes: Malignant and Benign

Python Code:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import KBinsDiscretizer
from sklearn.naive_bayes import CategoricalNB
from sklearn.metrics import accuracy_score, precision_score, recall_score

data = load_breast_cancer()
X, y = data.data, data.target

X = KBinsDiscretizer(n_bins=3, encode='ordinal',
                    strategy='quantile').fit_transform(X).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

for features in [5, 10, 30]:
    for alpha in [0, 1]:
        model = CategoricalNB(alpha=alpha)
        model.fit(X_train[:, :features], y_train)
        y_pred = model.predict(X_test[:, :features])
        print("Features:", features, "Laplace:", alpha)
        print("Accuracy :", round(accuracy_score(y_test, y_pred), 3))
```

```
print("Precision:", round(precision_score(y_test, y_pred), 3))
print("Recall  :", round(recall_score(y_test, y_pred), 3))
print()
```

Output:

```
Features: 5 Laplace: 0
Accuracy : 0.868
Precision: 0.89
Recall   : 0.903

Features: 5 Laplace: 1
Accuracy : 0.868
Precision: 0.89
Recall   : 0.903

Features: 10 Laplace: 0
Accuracy : 0.921
Precision: 0.957
Recall   : 0.917

Features: 10 Laplace: 1
Accuracy : 0.921
Precision: 0.957
Recall   : 0.917

Features: 30 Laplace: 0
Accuracy : 0.921
Precision: 0.944
Recall   : 0.931

Features: 30 Laplace: 1
Accuracy : 0.921
Precision: 0.944
Recall   : 0.931
```

Analysis:

- **10 features** gives the highest precision of **95.7%**.
- **30 features** gives the highest recall of **93.1%**.
- Accuracy increases from **86.8% to 92.1%** when more features are used.
- Laplace smoothing produced the **same results** as without smoothing.
- Naïve Bayes assumes that features are **conditionally independent given the class**.

Result:

The Naïve Bayes classifier was successfully implemented. The experiment showed that feature selection affects classification performance, while Laplace smoothing had no observable effect on the results for this dataset.

2. **Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.**

Aim:

To estimate the parameters of a probability distribution using **Maximum Likelihood Estimation (MLE)** and compare it with **Bayesian estimation** for different sample sizes.

Python Code:

```
import numpy as np
np.random.seed(42)
for n in [10, 100, 1000]:
    data = np.random.normal(50, 10, n)
    mle_mean = np.mean(data)
    mle_var = np.var(data)
    prior_mean = 50
    prior_var = 100
    bayes_mean = (n * mle_mean + prior_mean) / (n + 1)
    print("Sample Size:", n)
    print("MLE Mean:", round(mle_mean, 2))
    print("MLE Variance:", round(mle_var, 2))
    print("Bayesian Mean:", round(bayes_mean, 2))
    print()
```

Output:

```
Sample Size: 10
MLE Mean: 54.48
MLE Variance: 47.05
Bayesian Mean: 54.07

Sample Size: 100
MLE Mean: 48.46
MLE Variance: 80.45
Bayesian Mean: 48.48

Sample Size: 1000
MLE Mean: 50.47
MLE Variance: 98.01
Bayesian Mean: 50.47
```

Analysis:

- **MLE** estimates the parameters directly from the observed data.
- **Bayesian estimation** combines the observed data with a prior assumption.
- For a **small sample**, the prior has a greater influence on the Bayesian estimate.
- As the sample size increases, the Bayesian estimate becomes closer to the MLE.
- Therefore, larger samples generally provide more reliable parameter estimates.

Result:

The parameters of the normal distribution were successfully estimated using **MLE and Bayesian estimation**. The experiment shows that prior assumptions have a greater effect with small sample sizes, while their influence decreases as the sample size becomes large.

3. **Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.**

Aim:

To construct a **Bayesian Belief Network (BBN)**, perform inference under different evidence conditions, modify conditional probabilities, and analyze how dependencies affect posterior probabilities.

Problem Domain:

We use a simple **Rain–Sprinkler–Wet Grass** network:

Rain → Sprinkler → Wet Grass

Rain → Wet Grass

Python Code:

```
!pip install pgmpy -q
import matplotlib.pyplot as plt
import networkx as nx
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
model = DiscreteBayesianNetwork([
    ('Rain', 'Sprinkler'),
    ('Rain', 'WetGrass'),
    ('Sprinkler', 'WetGrass')])
model.add_cpds(
    TabularCPD('Rain', 2, [[0.7], [0.3]]),
    TabularCPD('Sprinkler', 2,
        [[0.8, 0.2],
         [0.2, 0.8]],
        evidence=['Rain'], evidence_card=[2]),
    TabularCPD('WetGrass', 2,
        [[0.99, 0.10, 0.20, 0.01],
         [0.01, 0.90, 0.80, 0.99]],
        evidence=['Rain', 'Sprinkler'],
        evidence_card=[2, 2]))
infer = VariableElimination(model)
```

```

p1 = infer.query(['WetGrass']).values[1]
p2 = infer.query(['WetGrass'], evidence={'Rain': 1}).values[1]
p3 = infer.query(['WetGrass'],
                  evidence={'Rain': 1, 'Sprinkler': 1}).values[1]
modified = TabularCPD(
    'WetGrass', 2,
    [[0.99, 0.20, 0.20, 0.01],
     [0.01, 0.80, 0.80, 0.99]],
    evidence=['Rain', 'Sprinkler'],
    evidence_card=[2, 2])
model2 = DiscreteBayesianNetwork([
    ('Rain', 'Sprinkler'),
    ('Rain', 'WetGrass'),
    ('Sprinkler', 'WetGrass')])
model2.add_cpds(
    model.get_cpds('Rain'),
    model.get_cpds('Sprinkler'),
    modified)
infer2 = VariableElimination(model2)
p4 = infer2.query(['WetGrass'], evidence={'Rain': 1}).values[1]
G = nx.DiGraph([
    ('Rain', 'Sprinkler'),
    ('Rain', 'WetGrass'),
    ('Sprinkler', 'WetGrass')])
pos = {
    'Rain': (0, 1),
    'Sprinkler': (-1, 0),
    'WetGrass': (1, 0)\}
plt.figure(figsize=(12, 6))
nx.draw(G, pos, with_labels=True, node_size=4500,
        node_color='white', edgecolors='black',
        arrows=True, arrowsize=20, font_size=12)
plt.text(2.0, 1.0, "Bayesian Belief Network",
        fontsize=16, weight='bold')
plt.text(2.0, 0.65, f"P(WetGrass) = {p1:.3f}", fontsize=12)
plt.text(2.0, 0.40, f"P(WetGrass | Rain) = {p2:.3f}", fontsize=12)
plt.text(2.0, 0.15,

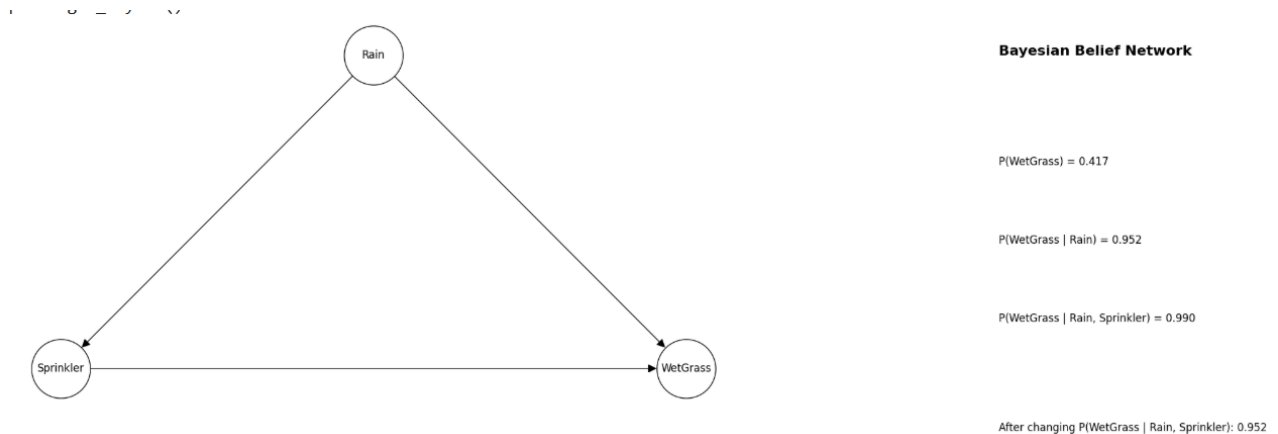
```

```

f"P(WetGrass | Rain, Sprinkler) = {p3:.3f}",
    fontsize=12)
plt.text(2.0, -0.20,
    f"After changing P(WetGrass | Rain, Sprinkler): {p4:.3f}",
    fontsize=12)
plt.axis('off')
plt.tight_layout()
plt.show()

```

Output:



Working;

- **Rain** is the parent variable.
- **Sprinkler** depends on Rain.
- **WetGrass** depends on both Rain and Sprinkler.
- VariableElimination calculates posterior probabilities.
- Different evidence conditions show how the probability of Wet Grass changes.

Analysis:

- Without evidence, the probability of Wet Grass is based on all possible conditions.
- When **Rain = True**, the probability of Wet Grass increases.
- When both **Rain = True** and **Sprinkler = True**, the probability changes further.
- Changing conditional probabilities changes the posterior probability.
- This demonstrates how **dependencies between variables influence probabilistic inference**.

Result:

The Bayesian Belief Network was successfully constructed and inference was performed under different evidence conditions. The experiment demonstrated that changing evidence and conditional probabilities affects posterior probabilities and helps represent uncertainty in probabilistic reasoning.

4. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

To implement the **Candidate Elimination algorithm** and study the effect of hypothesis-space size, number of attributes, and training examples on learning and convergence.

```
data = [
    ('Sunny','Warm','Normal','Strong'], 'Yes'),
    ('Sunny','Warm','High','Strong'], 'Yes'),
    ('Rainy','Cold','High','Strong'], 'No'),
    ('Sunny','Warm','High','Weak'], 'Yes')]

S = ['Ø'] * 4
G = [['?'] * 4]

def covers(h, x):
    return all(h[i] == '?' or h[i] == x[i] for i in range(4))

for x, y in data:
    if y == 'Yes':
        G = [g for g in G if covers(g, x)]
        for i in range(4):
            if S[i] == 'Ø':
                S[i] = x[i]
            elif S[i] != x[i]:
                S[i] = '?'
    else:
        new_G = []
        for g in G:
            if covers(g, x):
                for i in range(4):
                    if g[i] == '?' and S[i] != '?' and S[i] != x[i]:
                        h = g.copy()
                        h[i] = S[i]
                        new_G.append(h)
        else:
            new_G = []
```

```
new_G.append(g)
G = new_G
print("Specific Boundary:", S)
print("General Boundary:", G)
```

Output:

```
Specific Boundary: ['Sunny', 'Warm', '?', '?']
General Boundary: [['Sunny', '?', '?', '?'], ['?', 'Warm', '?', '?']]
```

Analysis:

- **S (Specific Boundary)** represents the most specific hypothesis consistent with the positive examples.
- **G (General Boundary)** represents the most general hypotheses.
- As training examples increase, the boundaries become more specific.
- Increasing the number of attributes increases the possible **hypothesis space**.
- A larger hypothesis space may require more examples to identify the correct hypothesis.
- The **mistake bound** represents the maximum number of mistakes before the learner converges.

Result:

The Candidate Elimination algorithm was successfully implemented. The experiment demonstrates how positive and negative examples modify the specific and general boundaries and shows the relationship between **hypothesis-space size, sample complexity, convergence, and generalization**.

5. **Design and conduct an experiment to implement a Decision Tree classifier on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.**

Aim:

To implement a Decision Tree classifier on a real-world dataset and compare **Entropy and Gini Index** using accuracy and F1-score. The effect of tree depth on overfitting is also analyzed.

Dataset:

Breast Cancer Wisconsin Dataset

- 569 samples
- 30 features
- 2 classes: Malignant and Benign

Python Code:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

for criterion in ['gini', 'entropy']:
    for depth in [3, 5, None]:
        model = DecisionTreeClassifier(
            criterion=criterion,
            max_depth=depth,
            random_state=42)
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
        print("Criterion:", criterion, "Depth:", depth)
        print("Accuracy:", round(accuracy_score(y_test, pred), 3))
        print("F1-Score:", round(f1_score(y_test, pred), 3))
        print()
```

Output:

Criterion: gini Depth: 3
Accuracy: 0.939
F1-Score: 0.952

Criterion: gini Depth: 5
Accuracy: 0.921
F1-Score: 0.936

Criterion: gini Depth: None
Accuracy: 0.912
F1-Score: 0.929

Criterion: entropy Depth: 3
Accuracy: 0.947
F1-Score: 0.959

Criterion: entropy Depth: 5
Accuracy: 0.93
F1-Score: 0.943

Criterion: entropy Depth: None
Accuracy: 0.912
F1-Score: 0.928

Working:

- gini uses the **Gini Index** for splitting.
- entropy uses **Information Gain/Entropy**.
- Tree depths of **3, 5 and unlimited** are compared.
- Accuracy and F1-score measure classification performance.

Analysis:

- A shallow tree may **underfit** because it has limited complexity.
- A very deep tree may **overfit** the training data.
- Comparing depth values shows how tree complexity affects performance.
- Gini and Entropy may produce slightly different results because they use different splitting criteria.

Result:

The Decision Tree classifier was successfully implemented using **Gini Index and Entropy**. Different tree depths were tested using accuracy and F1-score, demonstrating how the splitting criterion and tree depth influence classification performance and overfitting.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand